FP447 · Producto 2

Sistema de gestión de una cadena de hoteles

Implementación en Java con patrón MVC, colecciones, excepciones y pruebas JUnit

Autor: Yanmay Claros Portela (trabajo individual)

Archivo de entrega: producto2_FP447_YanmayClarosPortela

Enlace al vídeo	https://youtu.be/TW-eu7xvR8A
Repositorio GitHub	https://github.com/Yankpmaykp/proyectojava.git

1. Introducción

Este producto implementa en Java los diagramas diseñados en el Producto 1 (casos de uso y modelo estático de clases). El resultado es una aplicación de consola para el personal de una cadena hotelera que gestiona clientes, hoteles, habitaciones y reservas, siguiendo el patrón Modelo-Vista-Controlador. Los datos se almacenan en estructuras dinámicas en memoria; la persistencia en base de datos queda para la Fase 3.

El proyecto cumple las condiciones del Anexo 1: tres paquetes (vista, controlador, modelo), toda la interacción con el usuario en la vista, y acceso al modelo únicamente a través de la clase Controlador. Además se usan clases genéricas, colecciones elegidas según cada necesidad, excepciones personalizadas, pruebas con JUnit 5 y control de versiones con Git.

2. Trabajo realizado

2.1 Del diagrama de clases al código

Las clases del Producto 1 se han trasladado sin cambiar su diseño: Persona (abstracta) con sus subclases Cliente y Empleado; Hotel compuesto por Habitacion; Reserva compuesta por LineaReserva (una por habitación, con el precio por noche congelado en el momento de reservar); los enumerados CategoriaHabitacion, EstadoReserva y RolEmpleado; y las interfaces Identificable y Validable. Solo se han añadido las clases necesarias para la aplicación: Datos, Repositorio y las excepciones.

2.2 Patrón MVC

Paquete	Clases	Responsabilidad
hoteles.vista	VistaPrincipal, VistaClientes, VistaHoteles, VistaReservas, Consola	Menús, lectura de datos y visualización. Solo Consola usa el teclado.
hoteles.controlador	Controlador	Único puente vista-modelo. Recibe primitivos y cadenas, y devuelve texto ya formateado.
hoteles.modelo	Entidades, Datos, Repositorio<T>, excepciones	Datos de la aplicación y toda la lógica que los modifica (altas, disponibilidad, reservas).

Una decisión de diseño importante es que el Controlador no devuelve objetos del modelo a la vista, sino cadenas ya formateadas. Así la vista no depende de ninguna entidad. La única clase del modelo que la vista importa es HotelesException, para capturar los errores y mostrar su mensaje al usuario.

Respecto al Producto 1 hay un cambio: allí dejé la comprobación de disponibilidad frente a otras reservas en el controlador. En esta fase la he movido a la clase Datos, porque el enunciado asigna a Datos todas las acciones que afectan a los datos, y deja al Controlador como simple intermediario.

2.3 Genéricos y elección de colecciones

La clase genérica Repositorio<T extends Identificable> encapsula el almacenamiento de cada entidad. El límite superior Identificable garantiza que todo elemento tiene getId(). Se reutiliza para clientes, hoteles, habitaciones y reservas, evitando cuatro implementaciones casi idénticas.

Colección	Dónde se usa	Por qué
TreeMap<Integer, T>	Repositorio<T>	Búsqueda por id en O(log n) y recorrido siempre ordenado por id, sin ordenar después.
ArrayList	Hotel.habitaciones, Reserva.lineas	Colecciones pequeñas que se recorren secuencialmente y conservan el orden de inserción.
HashSet<Integer>	Datos.idsOcupadas	Reúne las habitaciones ocupadas sin duplicados y comprueba pertenencia en O(1).
EnumMap	Recuento por categoría	Claves de un enumerado: más compacto y rápido que un HashMap, y ordenado por categoría.
Comparator + List.sort	Búsqueda de reservas	Resultados ordenados por fecha de entrada y, a igualdad, por id.

2.4 Gestión de excepciones

Los errores de negocio se modelan con una jerarquía de excepciones comprobadas que parte de HotelesException. Con ello la vista captura un único tipo y muestra el mensaje, pensado para el usuario.

Excepción	Cuándo se lanza
ValidacionException	Datos obligatorios vacíos, email inválido, salida anterior a entrada, personas fuera de capacidad.
ElementoNoEncontradoException	Se pide un cliente, hotel, habitación o reserva que no existe.
ElementoDuplicadoException	DNI de cliente repetido, número de habitación repetido en un hotel.
HabitacionNoDisponibleException	La habitación ya está reservada en esas fechas.
OperacionNoPermitidaException	Eliminar un cliente con reservas, modificar una reserva cancelada o finalizada.

Las operaciones que modifican una reserva restauran el estado anterior si falla alguna validación, de modo que un error nunca deja datos a medias. La vista, además, repite la pregunta cuando el usuario escribe un número o una fecha con formato incorrecto, sin llegar a lanzar excepciones al controlador.

2.5 Funcionalidades

- **Clientes:** listar, buscar por nombre o apellido, alta, modificar (Intro conserva el valor actual), eliminar (bloqueado si tiene reservas) y consultar sus reservas.
- **Hoteles y habitaciones:** mostrar, buscar por ciudad, habitaciones de un hotel, disponibles en unas fechas (con filtro por número de personas), recuento de disponibles y habitaciones por categoría. También altas de hotel y habitación.

- **Reservas:** flujo cliente, hotel, fechas, habitaciones disponibles, personas y creación con importe automático; consulta combinando cliente, hotel, fechas y estado; detalle; modificación; cancelación y finalización.

Regla de ocupación: una reserva ocupa el intervalo [entrada, salida), de modo que otra puede empezar el mismo día en que termina la anterior, y las reservas canceladas no ocupan habitación. El importe es la suma, por habitación, de precio por noche por número de noches.

Ejemplo real de salida (importe calculado automáticamente y rechazo de una habitación ocupada):

```
Reserva creada:
[3] Luis Pérez Ruiz | Hotel Mediterráneo | 2026-11-10 -> 2026-11-12 (2 noches) | 2 pers. |
CONFIRMADA | 210.00 €
      - Hab. 201 TRIPLE: 105.00 € x 2 noches = 210.00 €
ERROR: La habitación 102 no está disponible del 2026-11-01 al 2026-11-02.
```

2.6 Pruebas unitarias con JUnit 5

Se han elegido dos métodos con lógica de negocio y se han probado con varios casos cada uno. Las 12 pruebas pasan.

Método	Clase de prueba	Casos cubiertos
Reserva.calcularImporte()	ReservaTest (4)	Una habitación; varias habitaciones; el cambio de tarifa no altera una reserva ya hecha; reserva sin habitaciones.
Datos.crearReserva(...)	DatosTest (8)	Reserva correcta; solapamiento rechazado; entrada el día de salida de otra permitida; reserva cancelada libera la habitación; salida anterior a entrada; exceso de personas; cliente inexistente; modificación con choque deja la reserva intacta.

```
[        12 tests found           ]
[        12 tests successful      ]
[         0 tests failed          ]
```

2.7 Control de versiones con Git

El proyecto se ha desarrollado en un repositorio Git con 13 commits pequeños y firmados, cada uno correspondiente a un bloque coherente: librería de pruebas, modelo importado, excepciones, repositorio genérico, clase Datos, controlador, vistas, pruebas, scripts y documentación. El archivo .gitignore excluye los binarios generados (out/).

2.8 Estructura del proyecto y ejecución

```
producto2/
  src/hoteles/
    Main.java
    vista/        Consola, VistaPrincipal, VistaClientes, VistaHoteles, VistaReservas
    controlador/  Controlador
    modelo/       Persona, Cliente, Empleado, Hotel, Habitacion, Reserva, LineaReserva,
                  enums, Identificable, Validable, Datos, Repositorio, excepciones
  test/hoteles/modelo/   ReservaTest, DatosTest
  lib/                   junit-platform-console-standalone-1.10.2.jar
```

```
compilar.bat  ejecutar.bat  probar.bat  README.md  Autoevaluacion.md
```

Requisitos: JDK 17 o superior en Windows. Se compila con compilar.bat, se arranca con ejecutar.bat y las pruebas se lanzan con probar.bat. Al iniciar se cargan dos hoteles, siete habitaciones, tres clientes y dos reservas de ejemplo para poder probar todas las opciones sin teclear datos.

2.9 Flujo de una reserva a través del MVC

Para mostrar cómo colaboran las tres capas, este es el recorrido de la opción Nueva reserva:

Paso	Capa	Qué ocurre
1	Vista	VistaReservas pide cliente, hotel y fechas a través de Consola y llama al Controlador.
2	Controlador	habitacionesDisponibles(...) delega en Datos y devuelve las habitaciones libres como texto.
3	Modelo	Datos calcula las habitaciones ocupadas (HashSet) y filtra las del hotel que están en servicio.
4	Vista	El usuario indica personas y habitaciones; la vista llama a crearReserva(...).
5	Modelo	Datos valida cliente, hotel, fechas, capacidad y disponibilidad; crea la Reserva con sus líneas, calcula el importe y la guarda como CONFIRMADA.
6	Vista	Si hay error llega una HotelesException con un mensaje comprensible; si no, se muestra el detalle.

3. Dificultades encontradas

- Mantener la separación estricta del MVC: obligó a que el controlador devolviese texto formateado en lugar de objetos del modelo.
- Decidir dónde comprobar la disponibilidad. La solución fue concentrarla en Datos y reutilizar Reserva.seSolapaCon, con la regla [entrada, salida).
- Evitar estados inconsistentes al modificar una reserva: se guardan los valores anteriores y se restauran si falla la validación o hay solapamiento.
- Compilación en Windows: la ruta del proyecto contiene un espacio y el separador del classpath es ';', lo que se resolvió en los scripts compilar.bat, ejecutar.bat y probar.bat.

4. Puntos no realizados

No se ha implementado persistencia: los datos se pierden al cerrar la aplicación y se cargan unos datos de ejemplo al arrancar. Tampoco hay autenticación de empleados ni modificación de las habitaciones de una reserva ya creada (hay que cancelarla y crear otra). Son aspectos fuera del alcance de este producto.

5. Propuestas de mejora

- Persistir los datos con JDBC y el patrón DAO (Fase 3), sustituyendo Datos por la capa de acceso a datos.
- Autenticar al empleado y limitar operaciones según su rol (recepcionista o administrador).
- Permitir cambiar las habitaciones de una reserva y aplicar tarifas por temporada.
- Sustituir el menú de consola por una interfaz gráfica (Fase 5) reutilizando el controlador.

6. Autoevaluación

Criterio	Nivel	Puntos
Control de versiones con Git	Correcto y optimizado	1 / 1
Generación de código a partir de un diagrama de clases	Correcto y optimizado	2 / 2
Colecciones en Java	Óptimas y optimizadas	4 / 4
Patrón de diseño MVC	Correcto y optimizado	4 / 4
Pruebas unitarias con JUnit	Correcto y optimizado	1 / 1
Total		12 / 12

Nota propuesta: 20 sobre 20. Considero que el trabajo cumple todos los requisitos de la rúbrica en su nivel más alto: el código parte fielmente del diagrama del Producto 1; cada colección se ha elegido por una razón concreta (TreeMap, ArrayList, HashSet, EnumMap) y se ha justificado; el MVC es estricto, con una vista que no conoce las entidades; hay excepciones personalizadas; las 12 pruebas JUnit pasan e incluyen casos límite; y el historial de Git está dividido en commits pequeños y descriptivos. Los puntos no realizados (persistencia y autenticación) quedan fuera del alcance de este producto y se indican en el apartado 4.

7. Webgrafía

- Oracle. Java SE 21 API Documentation. https://docs.oracle.com/en/java/javase/21/docs/api/
- Oracle. The Java Tutorials: Generics. https://docs.oracle.com/javase/tutorial/java/generics/
- Oracle. The Java Tutorials: Collections. https://docs.oracle.com/javase/tutorial/collections/
- JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/
- Git. Documentation. https://git-scm.com/doc